

AULA 1 · 11 de agosto · U1, Linguagens e Paradigmas

Por que existem tantas linguagens

Abertura da disciplina, o território do mercado e o primeiro commit

CODING: LINGUAGENS E TÉCNICAS

Prof. Dr. Sebastião Rogério · Faculdade SENAC Pernambuco, ADS 2026.2



"Qual linguagem eu devo aprender?"

É a pergunta errada, e esta aula existe para trocá-la por uma melhor. Ao final do encontro a turma deve conseguir formular a pergunta certa sozinha.

O percurso de hoje

1

O contrato

Como o semestre funciona, o que será construído e como a avaliação é composta.

2

O território

O que o mercado usa de fato, segundo três fontes que chegam a respostas diferentes.

3

O critério

Como se escolhe uma linguagem quando a escolha é uma decisão de engenharia.

4

Laboratório

Ambiente, repositório, primeiro commit e escolha do domínio do projeto.

O bloco de laboratório ocupa metade do encontro

A disciplina é de prática dominante. A teoria aparece sempre ancorada no problema que o laboratório do dia resolve.

BLOCO 1

O contrato

O que a disciplina constrói, como cada encontro funciona e como a nota é composta.



O que esta disciplina constrói

Ao longo de 16 encontros, cada equipe desenvolve um único produto, em camadas, em vez de resolver exercícios descartáveis.

A sequência é sempre a mesma: primeiro o núcleo do problema em objetos, depois um servidor que expõe esses objetos, depois uma interface que consome esse servidor, e por fim a integração completa. Cada camada só é acrescentada quando a anterior tem teste passando.

O produto final é pequeno de propósito. O que a disciplina avalia é a qualidade das decisões técnicas, não o tamanho do sistema.



Exemplos de domínio possíveis

Biblioteca de bairro, oficina mecânica, petshop, escola de idiomas, controle de estoque, barbearia. Domínios pequenos, com regras que a equipe consegue explicar de cabeça.

As cinco unidades

Unidade	Tema	O que é entregue ao final	Aulas
U1	Linguagens e paradigmas	Repositório, domínio escolhido e um script refatorado em dois estilos	1 a 3
U2	Orientação a objetos	Núcleo do sistema em classes, com suíte de testes verde	4 a 8
U3	Back-end e API	Servidor com CRUD completo e dados persistidos em banco	9 e 10
U4	Front-end	Telas em HTML, CSS e JavaScript consumindo a própria API	11 a 13
U5	Full-stack e framework	Aplicação integrada, componentizada e apresentada	14 a 16

A entrega é incremental

Nenhuma equipe constrói tudo na última semana, e a correção também acontece ao longo do caminho.

Como funciona um encontro

10 min · Abertura

Retomada do que travou na aula anterior e pergunta sobre a leitura da véspera.

45 min · Teoria

Bloco curto e cronometrado, sempre ligado ao problema do laboratório do dia.

2h30 · Laboratório

Produção em equipe sobre o projeto, com acompanhamento em sala.

20 min · Fechamento

Duas equipes demonstram, todas fazem commit, e cada estudante registra o que travou.

A proporção não é acidental

Programação se aprende programando. O tempo de exposição teórica é deliberadamente menor que o tempo de teclado.

Três combinados

1

Notebook em toda aula

A disciplina não funciona apenas no caderno. Quem não puder trazer equipamento deve avisar na primeira semana para que se organize uma alternativa no laboratório.

2

Toda aula termina com commit

O repositório é o diário da equipe. Um encontro sem commit na data equivale a uma entrega não feita, mesmo que o trabalho tenha acontecido.

3

Usar IA é permitido e esperado

O que não é permitido é entregar código que não se sabe explicar. A defesa das entregas é oral e individual, e a pergunta pode recair sobre qualquer linha.

O terceiro combinado é o mais importante do semestre

Ele define o que a disciplina considera aprendizado: não é produzir código, é conseguir julgar código.

Como a nota é composta

Processo avaliativo	Instrumento	Peso	Data
1º processo	Prova teórico-prática	50%	22 de setembro
1º processo	Entrega parcial do projeto	50%	29 de setembro
2º processo	Prova teórico-prática	50%	1 de dezembro
2º processo	Projeto final e apresentação	50%	8 de dezembro

Média do semestre: soma dos dois processos dividida por dois

Aprovação com média igual ou superior a 7,0 e frequência de pelo menos 75%. A rubrica do projeto é publicada hoje, antes de qualquer produção.

BLOCO 2

O território

O que o mercado de fato usa, segundo três fontes que medem coisas diferentes e chegam a respostas diferentes.



Antes dos números, uma definição

Uma linguagem de programação é um conjunto de regras para descrever computações de forma que um humano consiga escrever e uma máquina consiga executar.

A tensão entre essas duas exigências explica quase tudo. O processador entende apenas sequências de números que representam instruções elementares. O humano precisa de algo legível, revisável por outra pessoa e passível de correção seis meses depois. Toda linguagem é uma proposta de acordo entre esses dois lados.

Linguagens mais próximas da máquina dão controle e velocidade, e cobram tempo de escrita e risco de erro. Linguagens mais próximas do humano dão produtividade, e cobram desempenho e previsibilidade. Não existe posição correta nesse eixo, existe posição adequada a um problema.

O mesmo programa em três níveis de abstração

Somar dois números e mostrar o resultado, escrito em três distâncias diferentes da máquina.

ASSEMBLY x86, próximo da máquina

```
mov eax, 2
add eax, 3
; converter para texto
; chamar a syscall de escrita
; tratar o retorno
; cerca de 20 linhas ao todo
```

C, nível intermediário

```
#include <stdio.h>

int main(void) {
    printf("%d", 2 + 3);
    return 0;
}
```

PYTHON, próximo do humano

```
print(2 + 3)

# uma linha
# sem declarar tipo
# sem compilar
# sem gerenciar memória
```



As três produzem exatamente o mesmo resultado

O que muda é quanto trabalho a linguagem faz por você e quanto controle ela tira de você. Essa troca é o assunto da próxima aula.

Fonte 1: o que as pessoas procuram

1º

Python, posição que ocupa há vários anos seguidos

18,53%

do índice, com folga sobre o segundo colocado

10º

Rust, subindo de 1,34% para 1,45%

?

O que o TIOBE mede

Volume de buscas na web por cursos, tutoriais e profissionais de cada linguagem. É um termômetro de atenção e de demanda declarada, e não de código efetivamente escrito.

!

Onde esse número engana

Uma linguagem muito buscada pode indicar gente aprendendo, ou gente com dúvida. Volume de busca não separa entusiasmo de dificuldade.

Fonte 2: quem escreve mais código aberto

Em agosto de 2025 o TypeScript ultrapassou Python e JavaScript e se tornou a linguagem com mais contribuidores no GitHub.

Linguagem	Contribuidores por mês	Crescimento no ano	Onde essa massa está
TypeScript	2,64 milhões	mais 66,6%	Front-end de porte, ferramentas, times grandes
Python	segundo colocado	mais 48,8%	Inteligência artificial, dados, automação
JavaScript	terceiro colocado	mais 24,8%	Web em geral, base instalada antiga



A explicação mais aceita para a virada

Código com tipos declarados é mais fácil de gerar, revisar e corrigir, tanto por pessoas quanto por assistentes automáticos. O tema volta no bloco sobre inteligência artificial.

GitHub Octoverse 2025.

Fonte 3: o que os profissionais dizem usar

66%

usaram JavaScript
no último ano

57,9%

usaram Python,
alta de 7 pontos

72%

admiram Rust, a
linguagem mais admirada



Mais usada não significa mais admirada

Rust lidera a admiração com 72%, seguida por Gleam com 70%, Elixir com 66% e Zig com 64%. Nenhuma delas aparece no topo de uso. Popularidade mede inércia e base instalada, admiração mede desejo.



O salto do Python é a maior variação do levantamento

Sete pontos percentuais em um ano. É a adoção de inteligência artificial aparecendo na estatística, já que praticamente toda a ferramentaria da área é escrita em Python.

Três fontes, três respostas diferentes

Nenhuma das três está errada. Elas respondem a perguntas diferentes, e por isso chegam a lugares diferentes.

Fonte	Aponta como líder	O que de fato mede	Viés conhecido
TIOBE	Python	Volume de buscas na web	Confunde interesse com dificuldade
Octoverse	TypeScript	Contribuidores em repositórios públicos	Ignora todo o código privado das empresas
Stack Overflow	JavaScript	Uso autodeclarado por quem responde	Amostra de quem frequenta o site e responde pesquisa



A competência que fica desta comparação

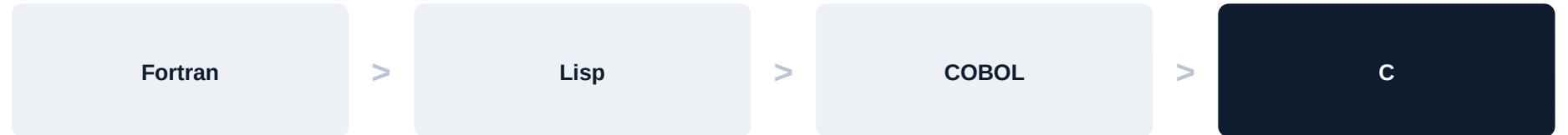
Antes de citar um ranking em uma reunião técnica, verifique o que ele mede. A pergunta "como esse número foi obtido" separa argumento de repetição.

Setenta anos de linguagens

Cada linguagem abaixo nasceu de um problema concreto que as existentes não resolviam bem.

1957 a 1972

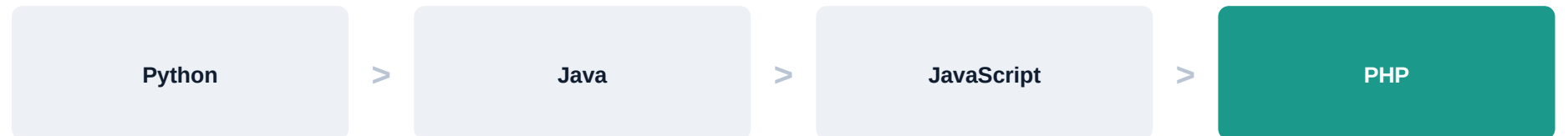
Provar que é possível



Fortran mostra que um compilador pode gerar código tão bom quanto o escrito à mão. C aparece porque escrever sistema operacional em Assembly era insustentável.

1991 a 1995

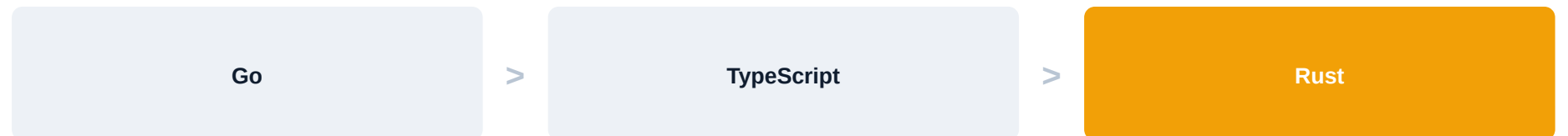
Produtividade e web



Python aposta em legibilidade. Java promete escrever uma vez e rodar em qualquer lugar. JavaScript é criado em dez dias para animar páginas.

2009 a 2015

Escala e segurança



Go nasce porque a compilação de C++ no Google levava tempo demais. TypeScript nasce porque bases grandes de JavaScript ficaram impossíveis de refatorar com segurança. Rust nasce por causa de erros de memória.

Linguagem velha não significa linguagem morta

Um contraexemplo útil contra a ideia de que só a tecnologia mais nova importa.

1

Onde ele ainda roda

Sistemas centrais de bancos, seguradoras, folhas de pagamento e órgãos públicos, escritos entre as décadas de 1960 e 1980 e nunca reescritos porque funcionam e a reescrita seria arriscada.

2

Por que não foi substituído

Trocar o sistema que processa transações de um banco significa aceitar risco de falha em algo que hoje não falha. O custo do erro é maior que o custo de manter.

3

O que isso ensina

Software vive muito mais do que o hype que o cercou no lançamento. A decisão técnica que você tomar em um projeto vai ser mantida por alguém, possivelmente por décadas.

A pergunta prática

Ao escolher uma tecnologia, vale perguntar quem vai manter isso daqui a cinco anos, e se será possível contratar essa pessoa.

O elefante na sala

84%

usam ou pretendem
usar IA para programar

46%

desconfiam da precisão
das respostas

3%

dizem confiar muito
no que a IA produz



A adoção subiu e a confiança caiu 11 pontos em um ano

Quase todo mundo usa e quase ninguém confia. Essa contradição descreve o mercado de trabalho atual: gerar código deixou de ser o gargalo, e julgar código passou a ser.



A consequência para esta disciplina

Um estudante que não entende tipagem, escopo e estrutura de dados não consegue revisar o que a ferramenta gerou. Ele só consegue aceitar. É essa diferença que o semestre pretende produzir.

BLOCO 3

O critério

Escolher linguagem é decisão de engenharia. Decisão de engenharia tem critério declarado, e critério pode ser discutido.



A pergunta certa

Não é "qual é a melhor linguagem". É: dado este problema, este time e este prazo, qual opção custa menos para construir e manter?

A mudança parece pequena, mas ela troca uma disputa de preferência por uma conta. A primeira pergunta não tem resposta, porque melhor depende de para quê. A segunda tem resposta, ainda que discutível, porque nomeia as restrições.

Em qualquer entrevista técnica ou reunião de arquitetura, quem responde à primeira pergunta soa como iniciante. Quem responde à segunda soa como profissional, mesmo que a conclusão final seja a mesma.



Duas respostas que não sobrevivem a uma reunião

"É a que eu já sei" e "é a que está em alta". Ambas podem até ser verdadeiras como fato, mas nenhuma das duas é justificativa técnica.

Seis critérios de escolha

1

O problema

Cálculo numérico, página web, firmware de microcontrolador e aplicativo de celular não pedem a mesma ferramenta.

2

O ecossistema

Existe biblioteca madura para o que precisa ser feito? Esse fator costuma pesar mais do que a linguagem em si.

3

O time

Uma linguagem que ninguém domina custa meses de curva antes da primeira entrega útil.

4

O mercado local

É possível contratar essa competência na região? Sistema sem quem mantenha tem prazo de validade.

5

A manutenção

Código é lido muitas vezes mais do que é escrito. Otimizar para leitura costuma ser mais econômico do que otimizar para escrita.

6

As restrições

Exigência de cliente, norma do setor ou infraestrutura existente às vezes já decidiram a questão antes da conversa começar.

Três cenários para decidir em equipe

A

A padaria da esquina

O dono quer parar de anotar encomendas no caderno. Volume baixo, orçamento curto, um único desenvolvedor, e o sistema precisa funcionar no celular dele.

B

Uma fintech

Precisa processar cinco mil transações por segundo sem perder nenhuma. Erro custa dinheiro e auditoria. Time de vinte pessoas e exigência de rastreabilidade.

C

Uma pesquisadora

Quer prever a demanda de leitos hospitalares a partir de oito anos de dados. O programa vai rodar poucas vezes, precisa de bibliotecas estatísticas e de gráficos apresentáveis.

Escolha uma linguagem para cada cenário e escreva uma frase de justificativa

Não existe resposta única. Existe justificativa que cita um critério do slide anterior e justificativa que não cita nenhum.

O mesmo problema, três linguagens

Problema comum a qualquer sistema de venda: somar o total de um carrinho de compras.

PYTHON

```
itens = [
    {"preco": 19.9, "qtd": 2},
    {"preco": 5.5, "qtd": 4},
]

total = sum(
    i["preco"] * i["qtd"]
    for i in itens
)
```

JAVASCRIPT

```
const itens = [
  { preco: 19.9, qtd: 2 },
  { preco: 5.5, qtd: 4 },
];

const total = itens.reduce(
  (s, i) => s + i.preco * i.qtd,
  0
);
```

JAVA

```
var itens = List.of(
    new Item(19.9, 2),
    new Item(5.5, 4)
);

double total = itens.stream()
    .mapToDouble(i ->
        i.preco() * i.qtd())
    .sum();
```

=

Repare no que é igual nas três

Percorrer a lista, multiplicar preço por quantidade e acumular o resultado. O raciocínio é idêntico, o vocabulário é que muda. É por isso que a disciplina se chama Linguagens e Técnicas, no plural: a técnica é transferível.

O que não muda quando a linguagem muda

1

Lógica

Condição, repetição e decomposição do problema em partes menores. É o núcleo, e é idêntico em qualquer linguagem.

2

Estruturas de dados

Saber quando usar lista, dicionário, fila ou conjunto vale mais do que dominar a sintaxe de qualquer uma delas.

3

Nomear bem

A habilidade mais subestimada da profissão. Nome ruim é uma dívida que se paga em toda leitura futura do código.

4

Testar

Sem uma forma de provar que funciona, não se sabe que funciona. Apenas se espera que sim.

5

Ler código alheio

A maior parte do tempo profissional é gasta lendo, não escrevendo. Isso vale para código de colega e para código gerado por máquina.

6

Depurar

Formular hipótese, isolar a variável, confirmar ou descartar. É método científico aplicado a software.

A regra de bolso do semestre

Uma linguagem se aprende em semanas. Estas seis competências levam anos, e são exatamente as que uma ferramenta automática ainda não substitui.

BLOCO 4

Mãos no teclado

Ambiente configurado, repositório no ar, primeiro commit e domínio do projeto escolhido, tudo ainda hoje.



A stack da disciplina e a razão de cada escolha

1

Python

Base de orientação a objetos e do back-end. Entrega os conceitos em menos linhas, o que sobra em tempo de laboratório. É também a linguagem da automação e da inteligência artificial.

2

HTML, CSS e JavaScript

Não existe atalho para o front-end: é o que o navegador executa. Todo framework moderno acaba compilando para essas três tecnologias.

3

Java, em um encontro

Um bloco comparativo na aula 8, para sentir na prática o que muda com tipagem estática e interface explícita, sem gastar o semestre em configuração e código cerimonial.

A ementa pede Java, e ele está no plano

A escolha é de sequência, não de exclusão. Aprender orientação a objetos e a cerimônia do Java ao mesmo tempo faz o estudante confundir o conceito com o ritual. Primeiro o conceito, depois a comparação.

Parte A: o ambiente

Cinco passos. Quem travar por mais de dez minutos no mesmo erro deve pedir ajuda em vez de insistir sozinho.

1 Python 3.12 ou superior
Baixar em python.org/downloads. No Windows, marcar a opção "Add Python to PATH" durante a instalação. Conferir no terminal com o comando `python --version`

2 Visual Studio Code
Baixar em code.visualstudio.com e instalar as extensões Python e Portuguese (Brazil) Language Pack

3 Git
Baixar em git-scm.com e conferir a instalação com o comando `git --version`

4 Conta no GitHub
Criar em github.com com um nome de usuário apresentável em processo seletivo. Esse perfil funciona como portfólio público.

5 Identificar-se ao Git
Executar `git config --global user name` e `git config --global user email` com os mesmos dados da conta criada

Entregável: ambiente funcionando na máquina de cada integrante da equipe

Parte B: Git em cinco comandos

TERMINAL, DO ZERO AO PRIMEIRO COMMIT

```
# 1. criar a pasta do projeto e entrar nela
mkdir projeto-equipe && cd projeto-equipe

# 2. iniciar o repositório local
git init

# 3. escrever o README.md e preparar o envio
git add .

# 4. registrar a versão com uma mensagem honesta
git commit -m "primeiro commit: define o dominio do projeto"

# 5. conectar ao GitHub e enviar
git remote add origin <url-do-repositorio>
git push -u origin main
```



Mensagem de commit é comunicação, não formalidade

Ruim: "ajustes", "teste", "aaa", "agora vai". Boa: "adiciona validacao de CPF no cadastro de cliente". Daqui a seis semanas, quando algo quebrar, essas mensagens são o que permite descobrir onde o problema começou.

O que cada comando significa

Uma analogia que funciona bem: o Git como uma sequência de fotografias do projeto.

git add

Escolhe o que vai entrar na foto.
Nem tudo que está na pasta precisa ser registrado.

git commit

Bate a foto e dá um nome a ela. A partir daqui é possível voltar a esse ponto.

git push

Envia as fotos para o servidor, onde os colegas e o professor conseguem ver.

git status

Mostra o que mudou desde a última foto. É o comando mais usado no dia a dia.

Git não é backup

Backup guarda o estado atual. O Git guarda a história inteira, com autor e motivo de cada mudança. É essa história que permite investigar um defeito.

Parte C: escolher o domínio

Equipes de 3 a 4 pessoas. Os seis abaixo são sugestões, e a equipe pode propor outro domínio desde que atenda aos critérios.

Biblioteca de bairro

acervo, sócio, empréstimo, devolução, multa

Oficina mecânica

cliente, veículo, ordem de serviço, peça

Petshop

tutor, animal, banho e tosa, agenda

Escola de idiomas

aluno, turma, matrícula, frequência

Controle de estoque

produto, entrada, saída, estoque mínimo

Barbearia

cliente, profissional, horário, serviço

Critérios de um bom domínio



Um domínio adequado tem

De 4 a 6 entidades, não 20.

Regras que a equipe consegue explicar de cabeça, sem pesquisar.

Pelo menos uma regra genuinamente chata: multa por atraso, desconto por pacote, horário que não pode colidir. É essa regra que dá substância ao projeto.



Domínios a evitar

Rede social, marketplace ou clone de aplicativo de entrega: escopo que não fecha em um semestre.

Domínio que ninguém da equipe conhece na prática.

Qualquer coisa que dependa de dado sigiloso real, de pessoas ou de empresas.

A regra chata é o coração do projeto

Um sistema que só cadastra e lista não exerce nenhum conceito de orientação a objetos. É a exceção da regra de negócio que obriga a modelar bem.

O que precisa estar pronto ao final da aula

1

Repositório público no GitHub

Nome no padrão ads-2026-2-dominio. Todos os integrantes adicionados como colaboradores.

2

README.md preenchido

Domínio escolhido, nomes da equipe e um parágrafo explicando que problema o sistema resolve.

3

Pelo menos um commit por integrante

Feito da própria máquina. É assim que se confirma que o ambiente de todos está funcionando.

4

Link entregue no ambiente virtual

Um envio por equipe, até as 23h59 de hoje.

Entregável: se o link for aberto e em 30 segundos não ficar claro o que o sistema faz, o README ainda não está pronto

Onde esta disciplina fica dentro do curso

Nenhuma disciplina do curso é uma ilha, e esta em particular alimenta várias outras.

1

O que vem antes

Lógica de programação e algoritmos deram a base de condição, repetição e estrutura de dados. Esta disciplina assume esse conhecimento e o organiza em torno de objetos.

2

O que corre em paralelo

Banco de dados fornece a camada de persistência que a unidade 3 vai usar. Engenharia de requisitos ensina a descobrir o que o sistema deve fazer, o que aqui já chega pronto na forma de domínio escolhido.

3

O que vem depois

Desenvolvimento web, mobile e as disciplinas de arquitetura partem do pressuposto de que orientação a objetos e integração entre camadas estão resolvidas. É o que se constrói aqui.

O projeto do semestre pode continuar vivo depois

Equipes que quiserem podem levar o mesmo domínio para as disciplinas seguintes, o que costuma render um portfólio mais consistente do que projetos desconexos.

Como pedir ajuda de um jeito que funciona

Vale para o professor, para os colegas, para fóruns e também para assistentes de inteligência artificial. Uma pergunta bem formulada costuma se responder sozinha no meio do caminho.

- 1 Diga o que você queria que acontecesse, e o que aconteceu no lugar.
- 2 Cole a mensagem de erro inteira, e não apenas a última linha ou uma descrição do que ela parecia dizer.
- 3 Informe o que já foi tentado. Isso evita que a ajuda comece por onde você já passou.
- 4 Reduza o problema ao menor exemplo que ainda falha. Em boa parte dos casos, o defeito aparece durante essa redução.
- 5 Diga a versão da linguagem e o sistema operacional. Muitos erros de instalação dependem só disso.

Referência clássica sobre o assunto: How To Ask Questions The Smart Way, de Eric S. Raymond.

Erros mais comuns do primeiro dia

1

No Windows: esquecer o Add to PATH

O sintoma é o terminal responder que python não é reconhecido como comando. A correção é reinstalar marcando a opção, ou acrescentar o caminho manualmente às variáveis de ambiente.

2

Commit feito sem se identificar ao Git

O commit aparece com autor desconhecido ou com o e-mail errado, e não conta como participação. Conferir com `git config --global user.email` antes do primeiro envio.

3

Repositório criado como privado

Se o repositório não for público, ninguém de fora consegue abrir o link entregue. Conferir abrindo a URL em uma janela anônima do navegador.

4

Apenas uma pessoa da equipe faz tudo

O histórico do Git mostra quem escreveu o quê. Uma equipe com commits de um único integrante gera conversa individual na aula seguinte.

O QUE FICA DESTA AULA

A pergunta não é qual linguagem aprender. É que problema se quer resolver.

- Os três rankings de mercado discordam porque medem coisas diferentes. Verificar o que um número mede é parte do trabalho técnico.
- Toda linguagem nasceu de uma dor concreta. Entender a dor explica as decisões de projeto da linguagem.
- Escolher tecnologia é decisão de engenharia, com critérios declarados e discutíveis.
- A técnica é transferível entre linguagens. A sintaxe é detalhe de sotaque.

Próxima aula: Paradigmas de programação, imperativo, funcional e orientado a objetos

Leitura para a próxima aula: Collection Pipeline, de Martin Fowler, apenas a introdução e a seção sobre map.



Referências desta aula

LIVROS

SEBESTA, Robert W. Conceitos de Linguagens de Programação. 11. ed. Porto Alegre: Bookman, 2018.

Bibliografia básica do PPC. Os capítulos 1 e 2 cobrem o panorama histórico e os critérios de avaliação de linguagens vistos hoje.

NYSTROM, Robert. Crafting Interpreters. Genever Benning, 2021.

Disponível gratuitamente e por inteiro no site do autor. Leitura para quem quiser entender como uma linguagem é construída por dentro.

HUNT, Andrew; THOMAS, David. O Programador Pragmático. 2. ed. Porto Alegre: Bookman, 2021.

Sobre o ofício e não sobre sintaxe. O capítulo sobre controle de versão dialoga com o laboratório de hoje.

XAVIER, Gley Fabiano Cardoso. Lógica de programação. São Paulo: Editora Senac São Paulo, 2018.

Bibliografia complementar do PPC. Apoio para quem sentir falta de base antes da unidade de orientação a objetos.

ARTIGOS, BLOGS E DOCUMENTAÇÃO

Joel Spolsky, The Law of Leaky Abstractions (leitura obrigatória para a aula 2)

<https://www.joelonsoftware.com/2002/11/11/the-law-of-leaky-abstractions/>

Stack Overflow Developer Survey 2025, seção Technology

<https://survey.stackoverflow.co/2025/technology>

GitHub Octoverse 2025, relatório anual

<https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>

Peter Norvig, Teach Yourself Programming in Ten Years

<https://www.norvig.com/21-days.html>

Paul Graham, Beating the Averages, sobre o paradoxo de Blub

<https://paulgraham.com/avg.html>

Crafting Interpreters, livro completo e gratuito

<https://craftinginterpreters.com/>